

I think this is the point where you should stop thinking of it as "a project" and start thinking of it as a **software product**.

Your roommate's projects feel impressive because they're architected like products, not assignments. Yours should be the same.

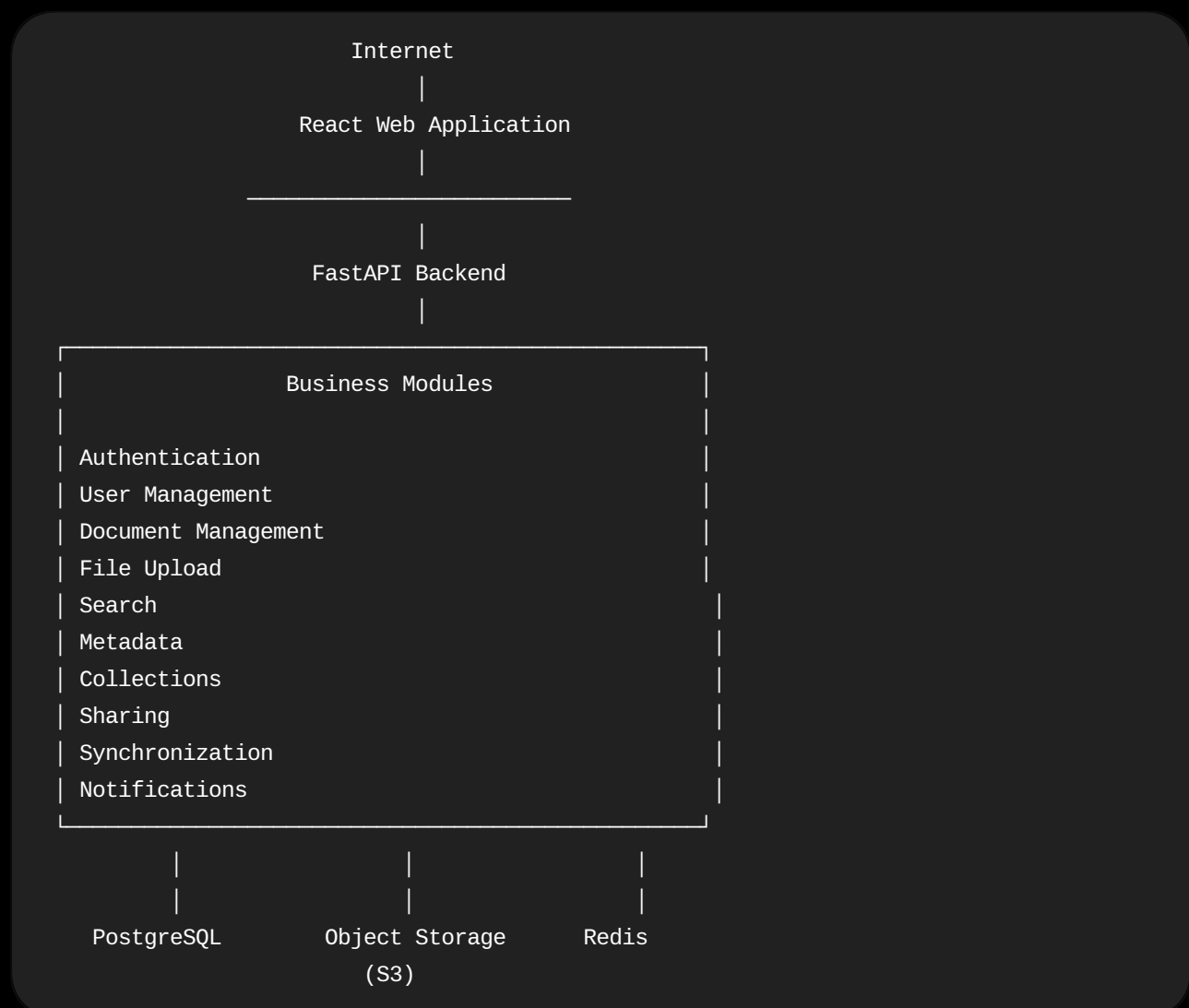
I would strongly recommend using technologies that are common in industry rather than the newest trendy framework. Recruiters care more about architecture and engineering decisions than whether you used the latest JavaScript library.

Overall Architecture

I would use a **modular monolith** for V1.

Not microservices.

Microservices are overkill for a single developer.



Later, if the project grows, individual modules like Search or Sync can be extracted into separate services.

Complete Tech Stack

Frontend

React

TypeScript

Tailwind CSS

React Router

TanStack Query

React Hook Form

Zod

Why?

- Industry standard
 - Great ecosystem
 - Type safety
 - Easy to scale
-

Backend

FastAPI

Python

SQLAlchemy

Alembic

Pydantic

Background Tasks

FastAPI is one of the best choices for this kind of application because it's fast, clean, and excellent for APIs.

Database

PostgreSQL

You will learn

- Indexes
- Constraints
- Foreign Keys
- Joins
- Transactions
- Full Text Search

This project will push your SQL skills much further than simple CRUD apps.

Storage

Don't store PDFs inside PostgreSQL.

Instead

PDF

↓

S3 Bucket

↓

Store only

filename

url

metadata

owner

hash

inside PostgreSQL

Even if you use MinIO locally, you'll be learning an architecture used in production.

Desktop App

I recommend

Tauri

instead of Electron.

Why?

Electron:

- Huge RAM usage

Tauri:

- Tiny
- Rust backend
- Uses native WebView

The desktop app should

- Watch Downloads
 - Detect new files
 - Upload files
 - Sync
 - Show notifications
-

Mobile

Android

Kotlin

Jetpack Compose

You don't need Flutter because your goal is to learn native Android development as well.

Authentication

JWT

Refresh Tokens

Role Based Access

Multiple devices

Sessions

Search

Start with

PostgreSQL Full Text Search

Don't jump to Elasticsearch.

Later

User searches

↓

API

↓

PostgreSQL Index

↓

Results

OCR

Tesseract OCR

This is practical, open source, and works well for scanned documents.

Background Jobs

Redis

Celery (or RQ/Dramatiq if you prefer a lighter stack)

Example

Upload PDF

↓

Queue OCR

↓

Extract text

↓

Index text

↓

Generate preview

↓

Notify User

Notifications

WebSockets

Real-time

or

Firebase Cloud Messaging

for Android.

Database Design

You'll probably have something like

Users

Devices

Documents

Folders

Collections

DocumentVersions

Tags

Permissions

AuditLogs

Notifications

Sessions

Uploads

OCRData

SyncEvents

That's already a very realistic schema.

Folder Structure

backend/

auth/

users/

documents/

storage/

search/

sync/

notifications/

sharing/

database/

models/

api/

frontend/

desktop/

android/

infra/

docs/

Notice how everything is separated.

Features That Demonstrate Engineering

Don't focus on quantity.

Focus on features that make interviewers ask questions.

For example

1. Automatic Sync Engine

Laptop edits document

↓

Backend stores version

↓

Phone gets update

↓

Conflict resolved

That alone demonstrates distributed systems concepts.

2. Download Watcher

Downloads/

↓

File Created

↓

Background Service

↓

Popup

↓

Upload

Very cool.

3. Duplicate Detection

Use SHA256

Same hash

↓

Duplicate

This is something real systems do.

4. Version Control

Resume

↓

Resume v2

↓

Resume v3

↓

Restore older version

5. Permission System

Owner

Editor

Viewer

Shared Links

Expiry

Read-only

6. Audit Logs

Document uploaded

↓

Edited

↓

Shared

↓

Downloaded

↓

Deleted

Every enterprise application has audit logs.

Things You'll Learn

Backend

REST APIs

Authentication

Authorization

Dependency Injection

Validation

Caching

Background Jobs

Queues

Rate Limiting

Pagination

Filtering

File Uploads

Streaming

Databases

Normalization

Indexes

Transactions

Constraints

Query Optimization

Full Text Search

Distributed Systems

Sync

Conflict Resolution

Versioning

Retries

Event Queues

Idempotency

Caching

Security

JWT

Encryption

Password Hashing

Signed URLs

File Validation

Access Control

HTTPS

Storage

Object Storage

Buckets

CDN concepts

Pre-signed URLs

Chunk Uploads (optional)

Desktop

Native Notifications

Background Services

Filesystem Events

Auto Updates

Android

Storage APIs

Share Sheet

Background Work

Notifications

Offline Cache

Things You Should Study Before Coding

I'd make this your learning roadmap:

Priority	Topic	Why it matters
★★★★★★	PostgreSQL (advanced)	Core data model and queries
★★★★★★	FastAPI	Main backend framework
★★★★★★	Authentication (JWT + Refresh Tokens)	User accounts and security
★★★★★★	React + TypeScript	Web frontend
★★★★★☆	Object Storage (S3/MinIO)	File storage architecture
★★★★★☆	Background jobs (Redis + Celery)	OCR, indexing, sync
★★★★★☆	Database schema design	Relationships and constraints
★★★★★☆	Filesystem watching	Desktop capture feature
★★★★☆☆	OCR (Tesseract)	Searchable scanned documents
★★★★☆☆	Android Share Sheet	Mobile capture
★★★☆☆☆	WebSockets	Real-time updates

One thing I'd change from your original vision

Initially, you wanted to support **email, WhatsApp, phone, laptop, Drive, etc.** Don't build every integration in V1.

A good V1 should solve **one core problem extremely well.**

I'd define the MVP as:

- Web application
- Desktop companion that watches the Downloads folder
- Android app that shares documents into the vault
- Cross-device synchronization
- Powerful metadata-based search
- Collections and secure sharing
- Version history

Once that foundation is solid, you can add Gmail import, Drive integration, OneDrive sync, automatic categorization, and other advanced features. Building the core architecture first will make those additions much easier later.

If you complete even this MVP with clean architecture, tests, documentation, and deployment, you'll have a portfolio project that demonstrates backend engineering, frontend development, database design, desktop programming, mobile integration, and distributed synchronization—all in one cohesive product. That's the kind of project that gives you substantial talking points in technical interviews.